

NATIONAL ECONOMICS UNIVERSITY

FACULTY OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE



BUILDING A LIVE AUCTION BIDDING SYSTEM: DESIGN AND OPERATION

Course: Network and Communication Technologies

Team Members: 11250640 - Nguyen Ngoc Anh

11255166 - Pham Ngoc Mai

11256961 - Nguyen Ha Phuong

Class: Data Science 67A

Date: September 20, 2026

Abstract

This report presents the design and implementation of a Live Online Auction Bidding prototype based on the Client-Server architecture. The primary objective is to simulate a real-time auction environment where multiple participants can connect, receive live updates, and submit bids simultaneously. The system is developed using Python TCP Sockets for reliable network communication and multi-threading techniques to handle concurrent client connections. To prevent race conditions and ensure data consistency, thread synchronization mechanisms (Locks) are applied to manage the shared “highest bid” variable. Additionally, the prototype integrates an automated 15-second count-down timer that resets upon receiving a valid higher bid and automatically declares the winner when the time expires. The experimental results, validated through various testing scenarios, demonstrate that the system successfully manages concurrent user interactions, accurately synchronizes shared states, and provides a stable real-time bidding experience.

The complete source code is publicly available on Github. For empirical evidence of the system’s performance, including stress test logs and UI behavior during race conditions, please refer to our full Test Result & Evidence

Acknowledgement

First and foremost, we would like to express our deepest gratitude to Dr. Tran Duc Minh, our lecturer, for his insightful teaching and for providing us with the opportunity to undertake this project. His guidance has been fundamental to our academic growth throughout this course.

We would also like to extend our sincere thanks to National Economics University for providing a dynamic learning environment and the necessary resources that greatly facilitated our research and academic journey.

Furthermore, we want to express our appreciation to all team members for their hard work, effective collaboration, and dedication throughout this project. Working together to handle complex multi-threaded synchronization and TCP socket communication has been an incredibly rewarding experience.

Despite our best efforts, we acknowledge that this report may still have certain limitations due to our practical inexperience. We warmly welcome any feedback and constructive criticism from you to help us improve in the future.

Contents

List of Figures	4
List of Listings	4
1 Introduction	5
2 Theoretical Background	6
2.1 TCP/IP Protocol and Client-Server Model	6
2.2 Multi-threading	6
2.3 Race Conditions and Resource Synchronization	6
3 System Architecture & Design	7
3.1 System Block Diagram	7
3.2 Application-layer Protocol & Security Hardening	7
3.3 Shared Data Structures and Resource Synchronization	8
3.4 Multi-threaded Operational Flow & Scalability Analysis	8
4 Implementation Details	10
4.1 Socket Initialization & Signal Handling	10
4.2 Safe Dictionary Iteration during Broadcasts	10
4.3 Critical Section Synchronization (Check-then-Act Pattern)	11
4.4 Timer Cancellation Race Mitigation	11
5 Testing & Results	13
5.1 Test Environment	13
5.2 Test case 1: Connection & Bid Validation	13
5.3 Test Case 2: Timer Reset Mechanism	14
5.4 Test Case 3: Concurrency & Stress Testing	15
5.5 Test Case 4: Fault Tolerance (Client Disconnection)	16
6 Conclusion	18
6.1 Achievements	18
6.2 Technical Challenges & Solutions	18
6.3 Future Work	18

List of Figures

1	Centralized Star Topology of the Auction System	7
2	Bid validation responses (ERROR and UPDATE)	14
3	The countdown timer explicitly resetting from 4s back to 15s.	15
4	Stress test results showing successful synchronization of 3,000 concurrent bids.	16
5	Server executing graceful degradation after a client disconnects unexpectedly.	17
6	Remaining clients successfully resolving the final winner	17

List of Listings

1	Socket Initialization	10
2	Safe dictionary iteration with lock	10
3	Check-then-Act synchronization for bidding	11
4	Mitigating timer cancellation race condition via version control	11

Chap 1

Introduction

This report presents the design and implementation of a Live Online Auction Bidding System, developed as our midterm project for the Network & Communication Technology course. Simulating a real-time auction provides a highly practical scenario to apply core computer networking concepts, specifically focusing on reliable data transmission and concurrent processing. To achieve this, our team focused on resolving the following key technical requirements as specified in the assignment:

- **TCP Socket Programming:** Establishing a reliable communication protocol between the central Server (Auctioneer) and multiple Clients (Bidders).
- **Multi-threaded Architecture:** Implementing multi-threading to handle multiple user connections simultaneously without blocking the system.
- **Shared Variable Synchronization:** Applying thread synchronization (Locks) to safely manage the shared "highest bid" variable, preventing race conditions during concurrent bid submissions.
- **Real-time Countdown Logic:** Developing a dynamic 15-second countdown timer that resets upon receiving a valid higher bid and concludes the auction automatically.

Chap 2

Theoretical Background

2.1 TCP/IP Protocol and Client-Server Model

The system follows the Client-Server model, in which a central server (the Auctioneer) maintains the authoritative auction state and multiple clients (the Bidders) connect to it to submit bids and receive updates.

TCP was chosen over UDP as the transport protocol because an auction cannot tolerate lost, duplicated, or reordered messages: a dropped bid or a missing price update would make the auction unfair. TCP provides connection-oriented, reliable, and in-order delivery through its three-way handshake, acknowledgements, and retransmission mechanisms. On the server side, a connection is established through the standard socket lifecycle (`socket()` → `bind()` → `listen()` → `accept()`), while each client simply calls `connect()`.

However, TCP operates as a continuous byte stream without message boundaries: several application messages may arrive in a single `recv()` call, or one message may be split across several calls. Consequently, the application layer must implement its own message framing mechanism (described in Section 3.2) to correctly separate and process each message.

2.2 Multi-threading

Multi-threading is essential for handling concurrent network tasks. The Server follows a **1-thread-per-client** model, assigning a dedicated thread to each connected bidder to process requests simultaneously. Meanwhile, the `client.py` utilizes a background thread exclusively to listen for server broadcasts, ensuring the main thread handling user inputs is never blocked.

2.3 Race Conditions and Resource Synchronization

Race conditions occur when multiple threads simultaneously modify shared resources, leading to data inconsistency. In this system, critical sections vulnerable to race conditions include: the **highest bid** variable, the **clients list**, and the **countdown timer**. To resolve this, we strictly implement **Mutex/Locks** to enforce mutual exclusion, ensuring only one thread can access and modify these shared states at any given time.

Chap 3

System Architecture & Design

3.1 System Block Diagram

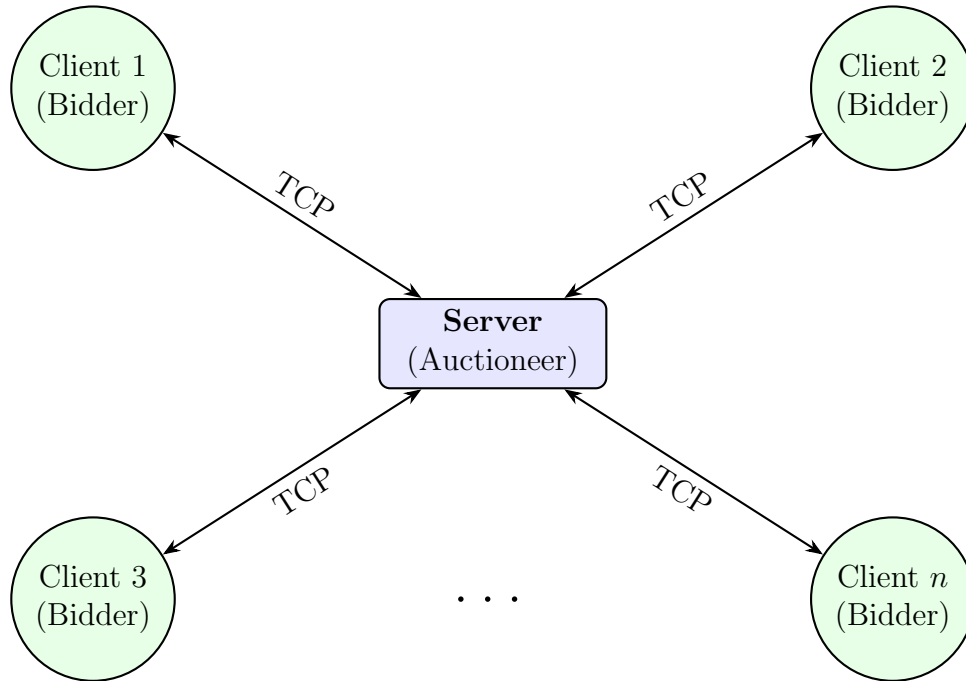


Figure 1: Centralized Star Topology of the Auction System

Client-Server architecture utilizing the TCP (Transmission Control Protocol) to ensure reliable, ordered, and error-checked delivery of data. The Server acts as the central Auctioneer, concurrently managing connections with n Clients (Bidders) in a star topology.

3.2 Application-layer Protocol & Security Hardening

The application layer relies on a custom text-based protocol using pipe-delimited (—) commands terminated by a newline character ($\backslash n$) for precise message framing.

- **Command Set:**

1. Client-to-Server: *JOIN* | *< username >*, *BID* | *< amount >* .
2. Server-to-Client: *WELCOME* | *< item >* | *< price >*, *UPDATE* | *< price >* | *< user >*, *WIN* | *< user >* | *< price >*, *ERROR* | *< reason >*, *INFO* | *< message >*.

- **Message Framing & Defense Mechanisms:** Because TCP is a stream-oriented protocol, consecutive `send()` calls can coalesce into single segments under high network loads (sticky packets). The `\n` delimiter enables the recipient's buffer to accurately isolate and parse discrete instructions.
- **Input Sanitization & Buffer Defense:** All incoming inputs are strictly validated. To prevent Out-of-Memory (OOM) attacks from clients sending infinite streams lacking delimiters, the socket stream is continuously decoded and flushed based on valid UTF-8 sequences.

3.3 Shared Data Structures and Resource Synchronization

To manage shared states safely without causing performance bottlenecks, the system employs a disciplined lock-separation strategy:

- `highest_bid`, `highest_bidder`, and the `auction_active` flag are strictly protected by a mutex named `price_lock`.
- The `clients` dictionary mapping active sockets to usernames is protected by a dedicated `room_lock`, preventing race conditions during insertions, removals, and broadcast iterations.
- The `timer_version` counter is protected by `price_lock`: it is incremented inside `reset_timer()`, which is always called within the lock, and re-validated under the same lock before the auction is closed.

3.4 Multi-threaded Operational Flow & Scalability Analysis

- **Server-side Flow:** The main thread continuously runs a timed `accept()` loop to handle incoming socket connections. Each successful connection spawns an independent daemon thread (`handle_client`). Each accepted bid (and the first JOIN) spawns a new daemon countdown thread tagged with the current `timer_version`; superseded threads exit on their next one-second tick.
- **Client-side Flow:** To prevent interface locking, the client immediately spawns a `receive_messages` daemon thread to process asynchronous `UPDATE` and `WIN` broadcasts, while the main thread blocks on standard input (`stdin`) to capture user `BID` actions
- **Scalability Trade-offs:** The Thread-per-Client model is optimal for small-to-medium-scale environments (e.g., our 30-client stress test setup), simplifying critical section management. However, for high-concurrency enterprise deployments (the C10K problem), this model experiences high context-switching overhead,

necessitating a transition to I/O Multiplexing (`select`, `epoll`) or asynchronous frameworks (`asyncio`).

Chap 4

Implementation Details

4.1 Socket Initialization & Signal Handling

To eliminate port binding failures (such as the *Address already in use* error caused by sockets stuck in `TIME_WAIT`), the `SO_REUSEADDR` flag is explicitly configured. Additionally, custom signal handlers for `SIGINT` and `SIGTERM` ensure uniform and graceful resource cleanup.

Listing 1: Socket Initialization

```
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server_socket.bind((HOST, PORT))
server_socket.listen(10)
```

4.2 Safe Dictionary Iteration during Broadcasts

When broadcasting messages via `send_to_all`, the server iterates over the shared `clients` dictionary. This loop is safely enclosed within `room_lock` to prevent a fatal `RuntimeError: dictionary changed size during iteration` if a client disconnects mid-broadcast.

Listing 2: Safe dictionary iteration with lock

```
def send_to_all(message, exclude=None):
    disconnected = []
    with room_lock:
        for sock, username in clients.items():
            if sock == exclude:
                continue
            if not send_message(sock, message):
                disconnected.append(sock)
        for sock in disconnected:
            clients.pop(sock, "unknown")
            sock.close()
```

4.3 Critical Section Synchronization (Check-then-Act Pattern)

The core concurrency defense lies in bid validation. Evaluating `amount > highest_bid` and updating `highest_bid` forms a classic Check-then-Act pattern. This compound operation must execute atomically within `price_lock`.

Listing 3: Check-then-Act synchronization for bidding

```
with price_lock:
    if not auction_active:
        reason = "Auction_has_already_concluded"
    elif amount > highest_bid:
        highest_bid = amount
        highest_bidder = username
        accepted = True
        reset_timer() # Timer re-allocation safely handled inside lock
    else:
        reason = f"Bid_too_low_(current:{highest_bid})"
```

Performance Optimization: Network I/O operations such as `send_to_all()` are intentionally executed outside `price_lock` to keep the critical section short and avoid blocking other bidding threads.

4.4 Timer Cancellation Race Mitigation

Each accepted bid calls `reset_timer()` inside `price_lock`, which increments `timer_version` and starts a new countdown thread tagged with that version. Superseded threads detect the mismatch and exit, so only the latest thread may close the auction.

However, in the initial implementation, the version was checked outside the lock, before `end_auction()` acquired it. If a bid was accepted in that short window, the stale thread would still close the auction immediately after the UPDATE was broadcast, denying other bidders their 15-second window. Checking `auction_active` alone cannot prevent this, because the bidding path never changes that flag.

The fix is to pass the thread's version into `end_auction()` and re-check it inside `price_lock`:

Listing 4: Mitigating timer cancellation race condition via version control

```
def end_auction(version):
    global auction_active
    with price_lock:
        if not auction_active or timer_version != version:
            return # already closed, or superseded by a newer bid
        auction_active = False
        winner, final_price = highest_bidder, highest_bid
        send_to_all(f"WIN|{winner}|{final_price}")
```

Since the version is both incremented and validated under the same lock, only two outcomes are possible: either the bid is accepted and the countdown restarts, or the auction closes first and the late bid is rejected. A bid can no longer be accepted and then immediately followed by the auction closing.

Chap 5

Testing & Results

5.1 Test Environment

- **Operating System:** MacOS
- **Interpreter:** Python 3.13 (Conda Environment)
- **Testing scripts:** Custom Python automation script (`stress_test.py`) with `threading.Barrier` for concurrent bot generation.

5.2 Test case 1: Connection & Bid Validation

- **Operation:** To verify that the server accurately validates incoming bids, rejecting invalid ones (lower or equal to the current price) and accepting valid ones.
- **Execution:** Two clients (phuong and mai) joined the room (starting price: 1,000,000 VND). phuong submitted a lower bid (10,000 VND) and an equal bid (1,000,000 VND); mai submitted a lower bid (100,000 VND). mai then bid 1,900,000 VND, and phuong bid 3,000,000 VND.
- **Result:** All three invalid bids were rejected with `ERROR` messages. Both valid bids were accepted and broadcast as `UPDATE` messages to every client in the room. When the countdown expired, phuong was declared the winner at 3,000,000 VND.
- **Evidence:**

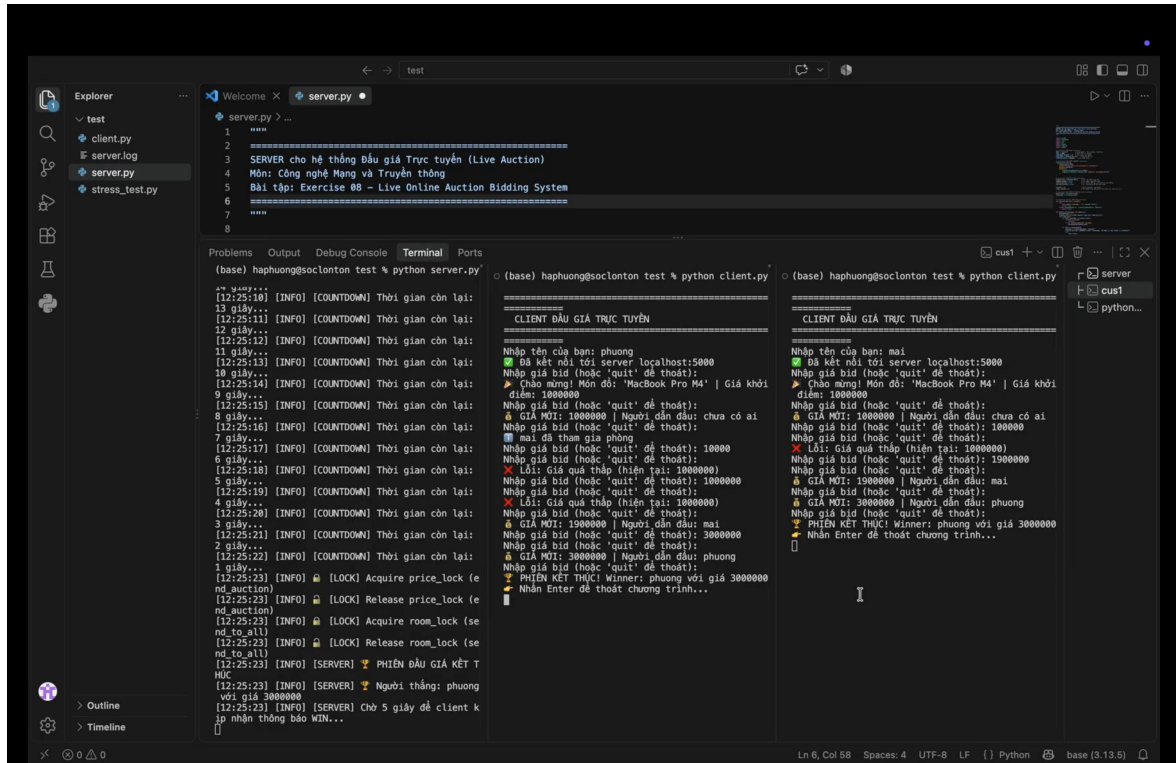


Figure 2: Clients receiving ERROR responses for invalid bids and UPDATE broadcasts for valid bids.

5.3 Test Case 2: Timer Reset Mechanism

- **Operation:** To ensure that the 15-second countdown timer accurately resets to its initial state upon receiving a valid bid.
- **Execution:** The countdown started when the first client joined. Client A bid 2,000,000 VND; when the countdown reached 4 seconds, Client B bid 3,000,000 VND.
- **Result:** The server instantaneously acquired the `price_lock`, registered the new bid, and reset the countdown timer back to 15 seconds. The console log clearly demonstrated the sequence jumping from 4 seconds back to 15 seconds.
- **Evidence:**

```
(base) haphuong@soclonten test % python server.py
[12:25:03] [INFO] [COUNTDOWN] Thời gian còn lại: 4 giây...
[12:25:04] [INFO] [COUNTDOWN] Thời gian còn lại: 3 giây...
[12:25:04] [INFO] [LOCK] Acquire price_lock (B ID)
[12:25:04] [INFO] [TIMER] Timer đã được RESET về 15s
[12:25:04] [INFO] [COUNTDOWN] Thời gian còn lại: 15 giây...
[12:25:04] [INFO] [LOCK] Release price_lock (B ID)
[12:25:04] [INFO] [SERVER] [✓] Chấp nhận BID 190000 từ 'mai'
[12:25:04] [INFO] [LOCK] Acquire room_lock (se nd_to_all)
[12:25:04] [INFO] [LOCK] Release room_lock (se nd_to_all)
[12:25:05] [INFO] [COUNTDOWN] Thời gian còn lại: 14 giây...
[12:25:06] [INFO] [COUNTDOWN] Thời gian còn lại: 13 giây...
[12:25:07] [INFO] [COUNTDOWN] Thời gian còn lại: 12 giây...
[12:25:08] [INFO] [LOCK] Acquire price_lock (B ID)
[12:25:08] [INFO] [TIMER] Timer đã được RESET về 15s
[12:25:08] [INFO] [COUNTDOWN] Thời gian còn lại: 15 giây...
[12:25:08] [INFO] [LOCK] Release price_lock (B ID)
[12:25:08] [INFO] [SERVER] [✓] Chấp nhận BID 300000 từ 'phuong'
[12:25:08] [INFO] [LOCK] Acquire room_lock (se nd_to_all)
[12:25:08] [INFO] [LOCK] Release room_lock (se nd_to_all)
[12:25:09] [INFO] [COUNTDOWN] Thời gian còn lại: 14 giây...

CLIENT ĐẦU GIÁ TRỰC TUYẾN
=====
Nhập tên của bạn: phuong
[✓] Đã kết nối tới server localhost:5000
Nhập giá bid (hoặc 'quit' để thoát):
> Chào mừng! Món đồ: 'MacBook Pro M4' | Giá khởi điểm: 1000000
Nhập giá bid (hoặc 'quit' để thoát):
[✓] GIÁ MỚI: 1000000 | Người dẫn đầu: chưa có ai
Nhập giá bid (hoặc 'quit' để thoát):
[✓] mai đã tham gia phòng
Nhập giá bid (hoặc 'quit' để thoát): 10000
Nhập giá bid (hoặc 'quit' để thoát):
[✗] Lỗi: Giá quá thấp (hiện tại: 1000000)
Nhập giá bid (hoặc 'quit' để thoát): 1000000
Nhập giá bid (hoặc 'quit' để thoát):
[✗] Lỗi: Giá quá thấp (hiện tại: 1000000)
Nhập giá bid (hoặc 'quit' để thoát):
[✓] GIÁ MỚI: 1900000 | Người dẫn đầu: mai
Nhập giá bid (hoặc 'quit' để thoát): 3000000
Nhập giá bid (hoặc 'quit' để thoát):
[✓] GIÁ MỚI: 3000000 | Người dẫn đầu: phuong
Nhập giá bid (hoặc 'quit' để thoát):

CLIENT ĐẦU GIÁ TRỰC TUYẾN
=====
Nhập tên của bạn: mai
[✓] Đã kết nối tới server localhost:5000
Nhập giá bid (hoặc 'quit' để thoát):
> Chào mừng! Món đồ: 'MacBook Pro M4' | Giá khởi điểm: 1000000
Nhập giá bid (hoặc 'quit' để thoát):
[✓] GIÁ MỚI: 1000000 | Người dẫn đầu: chưa có ai
Nhập giá bid (hoặc 'quit' để thoát): 100000
Nhập giá bid (hoặc 'quit' để thoát):
[✗] Lỗi: Giá quá thấp (hiện tại: 1000000)
Nhập giá bid (hoặc 'quit' để thoát): 1900000
Nhập giá bid (hoặc 'quit' để thoát):
[✓] GIÁ MỚI: 1900000 | Người dẫn đầu: mai
Nhập giá bid (hoặc 'quit' để thoát):
[✓] GIÁ MỚI: 3000000 | Người dẫn đầu: phuong
Nhập giá bid (hoặc 'quit' để thoát):
```

Figure 3: The countdown timer explicitly resetting from 4s back to 15s.

5.4 Test Case 3: Concurrency & Stress Testing

- **Operation:** To evaluate the effectiveness of the Mutex mechanism when subjected to a high volume of concurrent requests, aiming to expose potential race conditions.
- **Execution:** The `stress_test.py` script deployed 30 automated bots using a synchronization barrier. Each bot sent 100 bids simultaneously, resulting in 3,000 requests hitting the server within 3 seconds.
- **Result:** All 3,000 bids were processed in 3.27 seconds, with no server crash or unexpected disconnection. The highest bid sent by any bot (1,101,000 VND) matched the final price accepted by the server exactly (1,101,000 VND, leading bidder: `stress_bot_3`), and the script reported `PASS`. This indicates that `price_lock` correctly serialized all concurrent bid updates: no lower bid overwrote a higher one.
- **Evidence:**

```

1  """
2  SERVER cho hệ thống Bidding giá Trực tuyến (Live Auction)
3  Môn: Công nghệ Mạng và Truyền thông
4  Bài tập: Exercise 08 - Live Online Auction Bidding System
5  """
6
7
8
[12:32:39] [INFO] [LOCK] Release room_lock (disconnect cleanup)
[12:32:39] [INFO] [LOCK] Acquire room_lock (disconnect cleanup)
[12:32:39] [INFO] [SERVER] stress_bot_26 rời phòng. Còn lại: 5 client
[12:32:39] [INFO] [LOCK] Release room_lock (disconnect cleanup)
[12:32:39] [INFO] [LOCK] Acquire room_lock (disconnect cleanup)
[12:32:39] [INFO] [SERVER] stress_bot_9 rời phòng. Còn lại: 4 client
[12:32:39] [INFO] [LOCK] Release room_lock (disconnect cleanup)
[12:32:39] [INFO] [LOCK] Acquire room_lock (disconnect cleanup)
[12:32:39] [INFO] [SERVER] stress_bot_2 rời phòng. Còn lại: 3 client
[12:32:39] [INFO] [LOCK] Release room_lock (disconnect cleanup)
[12:32:39] [INFO] [LOCK] Acquire room_lock (disconnect cleanup)
[12:32:39] [INFO] [SERVER] stress_bot_17 rời phòng. Còn lại: 2 client
[12:32:39] [INFO] [LOCK] Release room_lock (disconnect cleanup)
[12:32:39] [INFO] [LOCK] Acquire room_lock (disconnect cleanup)
[12:32:39] [INFO] [SERVER] stress_bot_0 rời phòng. Còn lại: 1 client
[12:32:39] [INFO] [LOCK] Release room_lock (disconnect cleanup)
[12:32:39] [INFO] [LOCK] Acquire room_lock (disconnect cleanup)
[12:32:39] [INFO] [SERVER] stress_bot_15 rời phòng. Còn lại: 0 client
[12:32:39] [INFO] [LOCK] Release room_lock (disconnect cleanup)
[12:32:39] [INFO] [COUNTDOWN] Thời gian còn lại: 12 giây...
[12:32:40] [INFO] [COUNTDOWN] Thời gian còn lại: 11 giây...
[12:32:42] [INFO] [COUNTDOWN] Thời gian còn lại: 10 giây...
[12:32:43] [INFO] [COUNTDOWN] Thời gian còn lại: 9 giây...
[12:32:44] [INFO] [COUNTDOWN] Thời gian còn lại: 8 giây...
[12:32:45] [INFO] [COUNTDOWN] Thời gian còn lại: 7 giây...
[12:32:46] [INFO] [COUNTDOWN] Thời gian còn lại: 6 giây...
[12:32:47] [INFO] [COUNTDOWN] Thời gian còn lại: 5 giây...
[12:32:48] [INFO] [COUNTDOWN] Thời gian còn lại: 4 giây...
[12:32:49] [INFO] [COUNTDOWN] Thời gian còn lại: 3 giây...
[12:32:50] [INFO] [COUNTDOWN] Thời gian còn lại: 2 giây...
[12:32:51] [INFO] [COUNTDOWN] Thời gian còn lại: 1 giây...
[12:32:52] [INFO] [LOCK] Acquire price_lock (end auction)
[12:32:52] [INFO] [LOCK] Release price_lock (end auction)
[12:32:52] [INFO] [LOCK] Acquire room_lock (send_to_all)
[12:32:52] [INFO] [SERVER] PHÂN BẮC GIÁ KẾT THÚC
[12:32:52] [INFO] [SERVER] Người thắng: stress_bot_3 với giá 1101000
[12:32:52] [INFO] [SERVER] Cho 5 giây để client kịp nhận thông báo WIN...
[12:32:57] [INFO] [LOCK] Acquire room_lock (server shutdown)
[12:32:57] [INFO] [LOCK] Release room_lock (server shutdown)
[12:32:57] [INFO] [SERVER] Đã tắt server. Tạm biệt!
[12:32:57] [INFO] [SERVER] Port 5000 đã được giải phóng.
(base) haphuong@soclonton test %

```

```

python stress_test.py
[CLIENT] Đã đóng kết nối. Tạm biệt!
(base) haphuong@soclonton test % python stress_test.py

STRESS TEST - 30 clients x 100 bids
=====
Server: localhost:5000
Tổng số bid dự kiến: 3000

KẾT QUẢ STRESS TEST
=====
Thời gian test: 3.27 giây
Tổng số bid đã gửi: 3000
Giá cao nhất trong các bid đã gửi: 1101000
Giá cao nhất server công nhận: 1101000
Người dẫn đầu cuối cùng: stress_bot_3

PASS: Server công nhận đúng giá cao nhất!
-> Mutex hoạt động đúng, không có race condition.
(base) haphuong@soclonton test %

```

Figure 4: Stress test results showing successful synchronization of 3,000 concurrent bids.

5.5 Test Case 4: Fault Tolerance (Client Disconnection)

- **Operation:** To verify that the server safely performs graceful degradation when an active client disconnects during an ongoing auction.
- **Execution:** With three bidders in the room and about 13 seconds remaining, one client (mai) terminated the client program using **Ctrl+C**.
- **Result:** The server detected the closed connection, acquired **room.lock**, and removed the socket. The countdown continued without interruption (12s, 11s, 10s...), and the two remaining clients kept bidding until the auction concluded with na as the winner at 5,000,000 VND.
- **Evidence:**

```

(base) haphuong@soclon-ton test % python server.py
[20:40:57] [INFO] [LOCK] Acquire price_lock (BID)
[20:40:57] [INFO] [TIMER] Timer đã được RESET về 15s
[20:40:57] [INFO] [COUNTDOWN] Thời gian còn lại: 15 giây...
[20:40:57] [INFO] [LOCK] Release price_lock (BID)
[20:40:57] [INFO] [SERVER] Chấp nhận BID 4000000 từ 'na'
[20:40:57] [INFO] [LOCK] Acquire room_lock (send_to_all)
[20:40:57] [INFO] [LOCK] Release room_lock (send_to_all)
[20:40:58] [INFO] [COUNTDOWN] Thời gian còn lại: 14 giây...
[20:40:59] [INFO] [COUNTDOWN] Thời gian còn lại: 13 giây...
[20:41:00] [INFO] [COUNTDOWN] Thời gian còn lại: 12 giây...
[20:41:01] [INFO] [LOCK] Acquire room_lock (disconnect cleanup)
[20:41:01] [INFO] [SERVER] 'mai' đã rời phòng. Còn lại: 2 người đang đấu giá.
[20:41:01] [INFO] [LOCK] Release room_lock (disconnect cleanup)
[20:41:01] [INFO] [COUNTDOWN] Thời gian còn lại: 11 giây...
[20:41:02] [INFO] [COUNTDOWN] Thời gian còn lại: 10 giây...
[20:41:03] [INFO] [COUNTDOWN] Thời gian còn lại: 9 giây...
[20:41:04] [INFO] [COUNTDOWN] Thời gian còn lại: 8 giây...

(base) haphuong@192 test % python client.py
mai đã tham gia phòng
Nhập giá bid (hoặc 'quit' để thoát):
:
na đã tham gia phòng
Nhập giá bid (hoặc 'quit' để thoát): 2000000
Nhập giá bid (hoặc 'quit' để thoát):
:
GIÁ MỚI: 2000000 | Người dẫn đầu: hp
Nhập giá bid (hoặc 'quit' để thoát):
:
na đã tham gia phòng
Nhập giá bid (hoặc 'quit' để thoát):
:
GIÁ MỚI: 3000000 | Người dẫn đầu: mai
Nhập giá bid (hoặc 'quit' để thoát):
:
GIÁ MỚI: 4000000 | Người dẫn đầu: na
Nhập giá bid (hoặc 'quit' để thoát):
:
GIÁ MỚI: 4500000 | Người dẫn đầu: hp
Nhập giá bid (hoặc 'quit' để thoát):
:
GIÁ MỚI: 5000000 | Người dẫn đầu: na
Nhập giá bid (hoặc 'quit' để thoát):
:
PHIÊN KẾT THÚC! Winner: na với giá 5000000
Nhấn Enter để thoát chương trình.

(base) haphuong@192 test % python client.py
Nhập giá bid (hoặc 'quit' để thoát):
:
Chào mừng! Món đồ: 'MacBook Pro M4' | Giá khởi điểm: 1000000
Nhập giá bid (hoặc 'quit' để thoát):
:
GIÁ MỚI: 1000000 | Người dẫn đầu: chưa có ai
Nhập giá bid (hoặc 'quit' để thoát):
:
GIÁ MỚI: 2000000 | Người dẫn đầu: hp
Nhập giá bid (hoặc 'quit' để thoát):
:
GIÁ MỚI: 3000000 | Người dẫn đầu: mai
Nhập giá bid (hoặc 'quit' để thoát):
:
GIÁ MỚI: 4000000 | Người dẫn đầu: na
Nhập giá bid (hoặc 'quit' để thoát):
:
GIÁ MỚI: 4500000 | Người dẫn đầu: hp
Nhập giá bid (hoặc 'quit' để thoát):
:
GIÁ MỚI: 5000000 | Người dẫn đầu: na
Nhập giá bid (hoặc 'quit' để thoát):
:
PHIÊN KẾT THÚC! Winner: na với giá 5000000
Nhấn Enter để thoát chương trình.

```

Figure 5: Server executing graceful degradation after a client disconnects unexpectedly.

```

(base) haphuong@soclon-ton test % python server.py
price_lock (end_auction)
[20:41:25] [INFO] [LOCK] Release price_lock (end_auction)
[20:41:25] [INFO] [LOCK] Acquire room_lock (send_to_all)
[20:41:25] [INFO] [LOCK] Release room_lock (send_to_all)
[20:41:25] [INFO] [SERVER] PHIÊN KẾT THÚC
[20:41:25] [INFO] [SERVER] Người thắng: na với giá 5000000
[20:41:25] [INFO] [SERVER] Chờ 5 giây để client kịp nhận thông báo WIN
...
[20:41:30] [INFO] [LOCK] Acquire room_lock (server shutdown)
[20:41:30] [INFO] [LOCK] Acquire room_lock (disconnect cleanup)
[20:41:30] [INFO] [LOCK] Release room_lock (disconnect cleanup)
[20:41:30] [INFO] [LOCK] Release room_lock (server shutdown)
[20:41:30] [INFO] [LOCK] Release room_lock (disconnect cleanup)
[20:41:30] [INFO] [LOCK] Release room_lock (disconnect cleanup)
[20:41:30] [INFO] [SERVER] Đã tắt server. Tạm biệt!
[20:41:30] [INFO] [SERVER] Port 5000 đã được giải phóng.

(base) haphuong@192 test % python client.py
mai đã tham gia phòng
Nhập giá bid (hoặc 'quit' để thoát):
:
na đã tham gia phòng
Nhập giá bid (hoặc 'quit' để thoát): 2000000
Nhập giá bid (hoặc 'quit' để thoát):
:
GIÁ MỚI: 2000000 | Người dẫn đầu: hp
Nhập giá bid (hoặc 'quit' để thoát):
:
GIÁ MỚI: 3000000 | Người dẫn đầu: mai
Nhập giá bid (hoặc 'quit' để thoát):
:
GIÁ MỚI: 4000000 | Người dẫn đầu: na
Nhập giá bid (hoặc 'quit' để thoát):
:
GIÁ MỚI: 4500000 | Người dẫn đầu: hp
Nhập giá bid (hoặc 'quit' để thoát):
:
GIÁ MỚI: 5000000 | Người dẫn đầu: na
Nhập giá bid (hoặc 'quit' để thoát):
:
PHIÊN KẾT THÚC! Winner: na với giá 5000000
Nhấn Enter để thoát chương trình.

(base) haphuong@192 test % python client.py
Nhập giá bid (hoặc 'quit' để thoát):
:
Chào mừng! Món đồ: 'MacBook Pro M4' | Giá khởi điểm: 1000000
Nhập giá bid (hoặc 'quit' để thoát):
:
GIÁ MỚI: 1000000 | Người dẫn đầu: chưa có ai
Nhập giá bid (hoặc 'quit' để thoát):
:
GIÁ MỚI: 2000000 | Người dẫn đầu: hp
Nhập giá bid (hoặc 'quit' để thoát):
:
na đã tham gia phòng
Nhập giá bid (hoặc 'quit' để thoát):
:
GIÁ MỚI: 3000000 | Người dẫn đầu: mai
Nhập giá bid (hoặc 'quit' để thoát):
:
GIÁ MỚI: 4000000 | Người dẫn đầu: na
Nhập giá bid (hoặc 'quit' để thoát):
:
GIÁ MỚI: 4500000 | Người dẫn đầu: hp
Nhập giá bid (hoặc 'quit' để thoát):
:
GIÁ MỚI: 5000000 | Người dẫn đầu: na
Nhập giá bid (hoặc 'quit' để thoát):
:
PHIÊN KẾT THÚC! Winner: na với giá 5000000
Nhấn Enter để thoát chương trình.

```

Figure 6: Remaining clients successfully continuing the bidding process and resolving the final winner.

Chap 6

Conclusion

6.1 Achievements

The project successfully delivered a robust live online auction system using TCP sockets. Core networking requirements were met with high reliability: real-time bid broadcasting, multi-client concurrency handling, strict state synchronization via Mutex locks, and a self-healing countdown mechanism.

6.2 Technical Challenges & Solutions

- **Timer State Conflicts:** Each bid spawns a new countdown thread, so stale threads could close the auction prematurely. A `timer_version` token lets superseded threads exit, and the version is re-validated inside `price_lock` before closing, eliminating a residual check-then-act race (Section 4.4).
- **Socket Closure Exceptions (`Errno 9`):** Prematurely closing sockets before TCP buffers were fully flushed resulted in “Bad file descriptor”. This was mitigated by introducing a transmission delay (`time.sleep`) prior to the final shutdown sequence.
- **Dictionary Mutation during Iteration:** Iterating through the client dictionary during a broadcast while a client disconnected caused a (`RuntimeError`). This was resolved by maintaining a temporary list of broken sockets and purging them post-iteration inside a synchronized lock block.

6.3 Future Work

- Integrate a relational database (e.g., PostgreSQL/SQLite) for persistent user credentials and bidding history.
- Develop a robust Graphical User Interface (GUI) utilizing PyQt or Tkinter.
- Implement TLS encryption layers for secure payload transmission over public networks.
- Extend the core architecture to dynamically manage multiple parallel auction rooms.